VIT BHOPAL UNIVERSITY

School of Computing Science & Engineering

PROJECT REPORT

"Weather Prediction System"

(A Streamlit-based Weather Prediction System)

Submitted By: Bhavesh Wadhwani

Reg. No.: 25BAI10486

Branch: B.Tech – Computer Science & Engineering

Course Code: CSA2001

Course Name: Fundamentals of Artificial Intelligence and Machine learning

Submitted To: Dr Ashafaq Ahmad Najar

School of Computing Science & Engineering

Introduction

Weather plays a crucial role in our daily lives, influencing decisions related to travel, agriculture, business operations, and personal activities. However, weather conditions can change rapidly, and relying solely on manual observation or basic forecasts often leads to inaccurate planning and unexpected disruptions. Users frequently need timely and precise weather insights tailored to specific locations.

A Weather Prediction System is an intelligent application designed to address this challenge. It leverages technologies such as Python, APIs like OpenWeatherMap, and libraries/frameworks like Requests and Streamlit to collect, process, and display real-time or predictive weather data. By analyzing atmospheric parameters such as temperature, humidity, wind speed, and weather conditions, the system provides users with accurate and up-to-date forecasts.

This system enables users to quickly access reliable weather information through an interactive interface, helping them make informed decisions and plan their activities efficiently. By reducing uncertainty and improving accessibility to weather data, the project enhances user convenience and promotes smarter daily planning.

Problem Statement

Real-time weather information is essential for daily planning, travel, and safety, yet users often struggle to access accurate and location-specific forecasts quickly. Traditional sources may be slow, cluttered, or lack key details, making it difficult to interpret changing weather conditions efficiently. This results in inconvenience, poor decision-making, and unexpected disruptions in daily activities.

Project Objectives

• **Weather Interface Objective:** To design a clean, minimal, and user-friendly weather application using Python and Streamlit that allows users to easily input a city name and view real-time weather information.

• **Data Retrieval Objective:** To integrate reliable weather APIs (such as OpenWeatherMap) to fetch accurate and up-to-date weather data, including temperature, humidity, wind speed, and weather conditions.

• **System Functionality Objective:** To process and display weather data dynamically in an intuitive format, ensuring quick response time and clear visualization of key weather parameters.

• **User Experience Objective:** To develop a responsive and interactive front-end that enhances usability by presenting weather insights with visual elements like icons, cards, and structured layouts.

• **Error Handling Objective:** To implement robust error handling mechanisms for invalid inputs, API failures, and connectivity issues, ensuring smooth and reliable system performance.

• **Data Export Objective:** To enable the generation of downloadable weather reports (CSV/JSON) for selected cities, allowing users to store and analyze weather data offline if needed.

Functional Requirements

Functional requirements define the specific actions or tasks the system must perform.

1. Data Handling & Retrieval

- **User Input Handling** (city name input)
- **API Data Fetching** (retrieve weather data from OpenWeatherMap or similar API)
- ⎫ **Data Parsing & Processing** (extract temperature, humidity, wind speed, conditions)

2. Weather Prediction & Processing

- **Real-Time Weather Retrieval**
- **Weather Condition Analysis**
- **Temperature & Parameter Conversion** (e.g., Kelvin to Celsius if needed)
- **Icon/Condition Mapping** (map weather descriptions to visual icons)

3. Weather Output & Display

- **Current Weather Display** (temperature, condition, location)
- **Additional Parameters Display** (humidity, wind speed, feels like)
- **Structured Output Format** (cards, grids, or sections)
- **Clear and Readable Visualization**

4. User Interface (UI) Requirements

- **City Input Field**
- **Search / Get Weather Button**
- **Dynamic Weather Display Card**
- **Error Messages Display** (invalid city, API error, no input)
- **Responsive and Interactive Design**

5. System Reliability & Validation

- **Input Validation** (empty or incorrect city names)
- **API Error Handling** (invalid key, network issues, timeout)
- **Loading Indicator** (while fetching data)

Non-Functional Requirements

Non-functional requirements describe constraints and quality attributes of the system.

1. Performance

• **Response Time:** The system should fetch and display real-time weather data within 2–3 seconds of the user clicking the "Get Weather" button, depending on network conditions.

2. Usability (UX)

• **Error Handling:** The system must display clear and user-friendly error messages for invalid city names, missing input, API failures, or connectivity issues.

3. Reliability

• **Availability:** The Streamlit application should remain consistently available during usage and handle API failures or network interruptions gracefully without crashing.

4. Maintainability

• **Code Documentation:** All major functions and modules (such as API requests, data processing, and UI rendering) should include proper comments or docstrings explaining their functionality.

5. Portability

• **Browser Compatibility:** The application should function correctly across modern web browsers such as Chrome, Edge, and Firefox without UI distortion.

6. Scalability

• **Handling Multiple Requests:** The system should be capable of handling multiple user requests efficiently without significant performance degradation.

7. Security

• **API Key Protection:** Sensitive data such as API keys should be securely managed and not exposed publicly in the application interface or codebase.

Design Decisions & Rationale

Why Python?

- Fast for prototyping
- Streamlit simplifies UI development

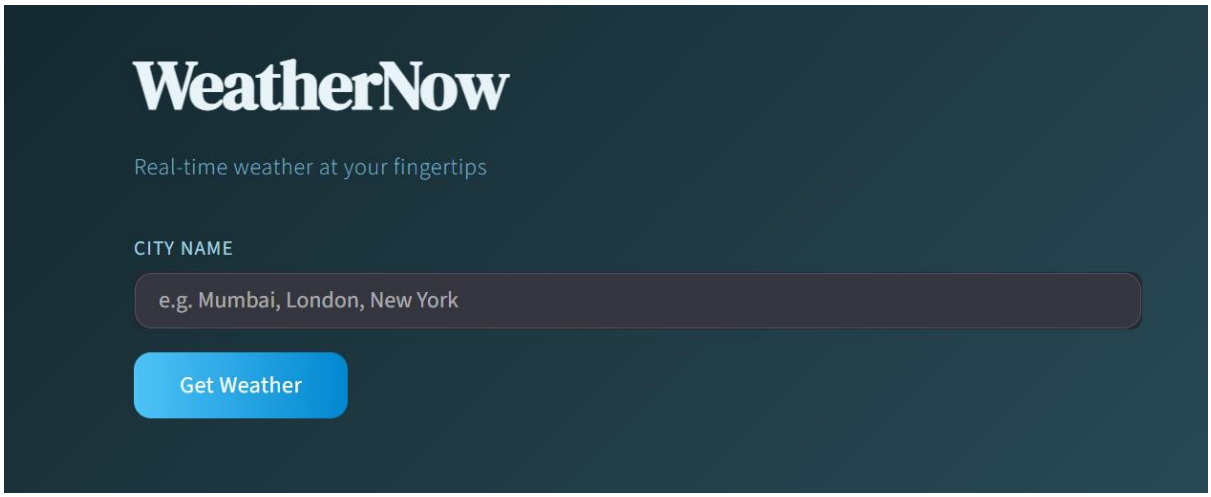
Why Streamlit?

- No frontend coding required
- Real-time auto-updating interface

Implementation Details

File	Purpose
1. Streamlit (`streamlit`)	• Used to build the **web-based user interface** of the weather application. • Helps create input fields, buttons, layouts, and display results interactively.
2 .Requests (`requests`)	Used to **send HTTP requests** to external APIs and fetch data.

Screenshots / Results



Challenges Faced

- API Integration & Data Dependency
- Data Parsing & Format Handling
- Streamlit Integration
- Real-Time Data Handling
- Error Handling & Validation
- UI/UX Design Complexity
- Network Dependency
- API Key Management & Security
- Scalability Issues

Learnings & Key Takeaways

- Understanding API Integration
- Working with Real-Time Data
- Streamlit Application Development
- Data Parsing and Processing
- Error Handling and Debugging
- UI/UX Design Principles
- Code Organization and Maintainability
- Importance of Security Practices
- Performance Optimization
- Real-World Application Development

Future Enhancements

- Model Improvement & Hybridization
- Data & Scalability
- User Experience (UX) & Features
- Implement a User Feedback Mechanism
- Implement live data updates

References

- OpenWeatherMap API Documentation
- Streamlit Documentation
- Requests Library Documentation
- Python Official Documentation
- Google Fonts (for UI Styling)